

RigGS: Rigging of 3D Gaussians for Modeling Articulated Objects in Videos

Yuxin Yao¹ Zhi Deng² Junhui Hou^{1*}

¹City University of Hong Kong ²University of Science and Technology of China

yuxinyao@cityu.edu.hk zhideng@mail.ustc.edu.cn jh.hou@cityu.edu.hk

<https://yaoyx689.github.io/RigGS.html>

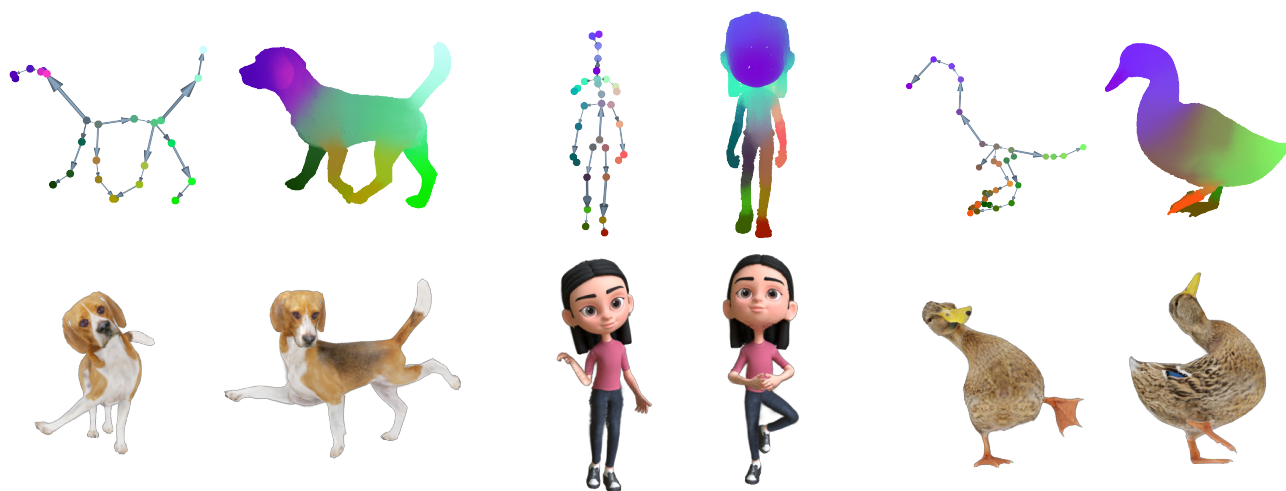


Figure 1. RigGS is a new and effective paradigm for automatically modeling articulated objects from 2D videos *without* any template prior. RigGS allows for easy editing and interpolation of object motion while supporting high-quality real-time rendering for these creative poses. We visualize the constructed skeleton (top-left), skinning weights (top-right), and edited new poses (bottom) for each object.

Abstract

This paper considers the problem of modeling articulated objects captured in 2D videos to enable novel view synthesis, while also being easily editable, drivable, and reposable. To tackle this challenging problem, we propose RigGS, a new paradigm that leverages 3D Gaussian representation and skeleton-based motion representation to model dynamic objects without utilizing additional template priors. Specifically, we first propose skeleton-aware node-controlled deformation, which deforms a canonical 3D Gaussian representation over time to initialize the modeling process, producing candidate skeleton nodes that are further simplified into a sparse 3D skeleton according to their motion and semantic information. Subsequently, based on the resulting skeleton, we design learnable skin deformations and pose-dependent detailed deformations, thereby easily deforming the 3D Gaussian representation to gen-

erate new actions and render further high-quality images from novel views. Extensive experiments demonstrate that our method can generate realistic new actions easily for objects and achieve high-quality rendering.

1. Introduction

Rigging involves extracting an animatable skeleton and binding it to a deformable object, which is crucial in movies, games, and AR/VR. The skeleton, composed of interconnected bones and control joints, offers a structured and semantically rich representation of motion. This process is essential for realistic animation, allowing for natural movement and deformation. It also simplifies tasks such as pose editing, motion interpolation, motion transfer, and dynamic animation creation.

Creating semantically plausible rigs automatically is challenging. Some methods use universal skeleton models from extensive datasets, like SMPL [17] for the human body, MANO [24] for the human hand, and SMAL [49] for animals. These models are essential for object recon-

*Corresponding author. This work was supported in part by the NSFC Excellent Young Scientists Fund 62422118, and in part by the Hong Kong RGC under Grants 11219324, 11202320, and 11219422.

struction, pose estimation, animation, etc. However, many dynamic objects, such as the human body with accessories, hands with personalized gloves, diverse animal species, deformable toys, or robots, cannot be standardized into a universal skeleton. Therefore, it is highly desired to develop a *template-free* model that can create a rig for any object with a skeletal structure.

Some methods extract skeletons from a 3D representation of the object [35, 36]. However, they rely on artist-designed skeletons as supervision and can only handle symmetric objects. Optimization-based methods [4, 13, 31] can extract the skeleton from any 3D model, but the skeletons obtained are often very dense. Other approaches combine motion information to extract motion-aware skeletons from point cloud sequences [1, 37]. However, due to limited 3D data availability, these methods often lack practicality. With the rise of neural rendering, some methods [33, 40] utilize 2D images or videos to obtain rigs for objects. However, they often require a predefined skeleton structure and only optimize joint positions. Other methods [11, 28] do not need a predefined skeleton tree but rely on existing techniques to extract skeletons from 3D models, making them dependent on the quality of the reconstructed geometry and skeleton extraction.

To address these challenges, we propose RigGS, a *template-free* automated rigging model that can extract 3D skeletons from monocular videos of moving objects and bind them to drive deformation. Firstly, we utilize a canonical 3D Gaussian shape representation that is deformed by a skeleton-aware node-controlled deformation field over time to initialize the modeling process, producing a set of skeleton-aware nodes. We then simplify the dense nodes using geometric, semantic, and motion information to achieve the final sparse 3D skeleton. Finally, we bind the canonical 3D Gaussians to the 3D skeleton to create a skeleton-driven dynamic model. Once trained, RigGS enables editing, motion interpolation, motion transfer, and animation of dynamic objects while supporting high-quality real-time rendering. Extensive experiments indicate that our RigGS can extract reasonable 3D skeletons, achieves rendering accuracy comparable to state-of-the-art methods, and allows for the flexible generation of new motions for the reconstructed dynamic objects.

In summary, we propose a novel *template-free* paradigm that can synthesize dynamic objects captured in 2D videos from novel views and facilitate editing to create new actions. The technical novelty of our approach lies in:

- we introduce a deformation field based on skeleton-aware nodes, combined with 3D Gaussians as the canonical shape representation, achieving simultaneous reconstruction of dynamic objects and obtaining candidate skeleton points;
- we propose a heuristic 3D skeleton construction algo-

rithm that considers geometric, semantic, and motion information; and

- we develop a skeleton-driven dynamic model with learnable skinning weights to bind the skeleton with the 3D Gaussian representation and a pose-dependent detailed deformation, allowing for flexible generation of new motions.

2. Related work

Geometric Representation and Rendering. Traditional 3D modeling utilizes textured meshes as a typical representation. With the emergence of neural implicit representations, many methods are employing them to model 3D objects or scenes [18, 30]. Recently, 3D Gaussian Splatting (3DGS) [10] has gained attention for its efficient and high-quality rendering capabilities, along with its ease of editing, gradually becoming an important representation in 3D vision. Numerous approaches [7, 15, 32, 41, 45, 46] have also emerged to address the task of novel view synthesis for dynamic objects or scenes captured by videos.

Prior-dependent Dynamic Modeling. Considering the challenges in skeleton extraction, various methods have leveraged category priors to establish specific parameterized models. For example, SMPL [17] is designed to represent human bodies, MANO [24] focuses on hand modeling, SMAL [49] extends to handle quadrupeds, and so on. These parameterized models have played important roles in tasks such as reconstruction, tracking, and animation within their domains [8, 19, 20, 42]. However, their representational capabilities are confined when dealing with shapes beyond their intended distributions, such as humans in intricate attire or animals from diverse categories. To enable broader applications, some methods do not utilize these parameterized models. CASA [34] establishes a database of animals, enabling category-agnostic skeletal animal reconstruction from monocular videos by retrieving and refining similar 3D models and skeletons from the database. Some methods take a pre-established skeletal tree structure as input, optimizing the positions of skeletal joints to model objects [33, 39, 43]. However, the pre-definition of the skeletal tree structure hampers the automation of skeleton creation.

Neural Bones for Dynamic Objects. To establish a deformation field for arbitrary objects, BANMo [38] introduces a representation using neural bones. It no longer relies on the traditional skeletal tree structure; instead, it employs learnable bones, involving spatial positions and transformations, to represent the deformation. These neural bones can be distributed on the surface, inside, or even outside of the object. Similar to some control point-based deformation fields [2, 7, 44], these bones do not carry semantic information. Following BANMo, BAGS [47] adopt diffusion prior and 3DGS to construct animatable model from a single ca-

sual video. DreaMo [27] further establishes the skeleton based on the learned bones. It derives the connections between bones through clustering, utilizing the reconstructed mesh and skinning weights. Compared with the traditional skeleton structure, it is easier to construct, but more difficult to create reasonable new actions.

Template-free Skeleton for Articulated Objects. Establishing a skeleton tree with a template-free algorithm for dynamic objects, thereby offering enhanced semantic meaning and editability is extremely challenging. Some methods utilize complete 3D representations, such as meshes, to achieve this [35, 36]. However, they can only handle meshes that have a symmetrical structure. Some methods focus on handling objects with articulated rigid structures [16, 25], such as laptops, glasses, or drawers. However, they cannot handle other types of dynamic objects, such as animals. Some methods take 2D images as input to build animatable models [11, 28]. They first reconstructed the dynamic representation and extracted the 3D mesh of the template shape, using the skeleton extraction method of the 3D model mentioned above to obtain the skeleton. Even though the skeleton will be refined in subsequent optimizations, they still rely on the quality of the 3D skeleton extraction and reconstruction of the template shape. Recently, SK-GS [29] utilizes 3D Gaussian Splatting and super-points to reconstruct dynamic objects and discovers the underlying skeleton model by treating super-points as rigid parts.

3. Proposed Method

Overview. Given a monocular video capturing continuous actions of an object, denoted as $\mathcal{I} = \{\mathbf{I}_t\}$ with $\mathbf{I}_t$ being the frame at time $t \in [0, 1]$, we aim to model the animatable dynamic object such that it can be rendered from novel viewpoints, and *more importantly, the object can be easily edited to create new actions*. To tackle this challenging problem, we propose an automated 3D skeleton construction method along with a skeleton-driven deformation model. As illustrated in Fig. 2, we initially model the dynamic object through 3D Gaussian splatting in conjunction with a skeleton-aware node-controlled deformation model, resulting in a 3D Gaussian-based canonical representation and nodes imbued with skeleton semantics. These skeleton-aware nodes allow for the generation of a dense and redundant skeleton model, which is further simplified and enhanced based on geometric, motion, and symmetry considerations. Finally, we introduce a skeleton-driven deformation model that finely fits the input video and generates new actions.

3.1. Initialization

At this stage, we initialize the 4D reconstruction of the dynamic scene captured in $\mathcal{I}$ using a canonical 3D Gaussian

representation that is deformed over time by skeleton-aware node-controlled deformation.

Canonical 3D Gaussian Representation. The 3D Gaussian representation [10] is a collection of attributed 3D Gaussians with each Gaussian G_i containing a center position μ_i , covariance matrix Σ_i , opacity σ_i and spherical harmonic coefficient sh_i . The covariance matrix Σ_i can be decomposed as $\Sigma_i = \mathbf{R}_i \mathbf{S}_i \mathbf{S}_i^T \mathbf{R}_i^T$ for optimization, where $\mathbf{R}_i$ is a rotation matrix represented by a quaternion $\mathbf{q}_i$, and $\mathbf{S}_i$ is a scaling matrix denoted by a 3D vector $\mathbf{s}_i$. Rendering an image from a specific viewpoint v_i involves projecting the 3D Gaussians onto a 2D plane, resulting in 2D Gaussians with projected means $\hat{\mu}_i$ and covariances $\hat{\Sigma}_i$. The color $\mathcal{C}(u)$ of the image pixel u can be calculated by

$$\mathcal{C}(u) = \sum_{i \in \mathcal{N}} T_i \alpha_i \mathcal{SH}(sh_i, v_i), \text{ where } T_i = \prod_{j=1}^{i-1} (1 - \alpha_j). \quad (1)$$

Here $\mathcal{SH}(\cdot, \cdot)$ is the spherical harmonic function, and α_i can be calculated by

$$\alpha_i = \sigma_i \exp\left(-\frac{1}{2}(u - \hat{\mu}_i)^T \hat{\Sigma}_i (u - \hat{\mu}_i)\right). \quad (2)$$

Therefore, a 3D scene is parameterized as $\mathcal{G} = \{G_i : \mu_i, \mathbf{q}_i, \mathbf{s}_i, \sigma_i, sh_i\}$, which will be adjusted adaptively during the optimization process.

Skeleton-aware Node-controlled Deformation. This module facilitates the temporal deformation of the canonical 3D Gaussian representation to model the dynamic scene. Specifically, the deformation of each 3D Gaussian at time t is achieved as follows:

$$\mu_i^t = \sum_{\mathbf{c} \in \mathcal{N}(\mu_i)} w_{\mu_i, \mathbf{c}} (\tilde{\mathbf{R}}_{\mathbf{c}}^t (\mu_i - \mathbf{c}) + \mathbf{c} + \tilde{\mathbf{t}}_{\mathbf{c}}^t), \quad (3)$$

where $\mathbf{C} = \{\mathbf{c}\}$ denotes the set of skeleton-aware nodes; $\tilde{\mathbf{R}}_{\mathbf{c}}^t \in \mathbb{R}^{3 \times 3}$ and $\tilde{\mathbf{t}}_{\mathbf{c}}^t \in \mathbb{R}^3$ are the rotation matrix and translation vector of node $\mathbf{c}$ at time t ; $\mathcal{N}(\mu_i)$ is the set of the k nearest points to μ_i in $\mathbf{C}$; the weight $w_{\mu_i, \mathbf{c}}$ is defined as

$$w_{\mu_i, \mathbf{c}} = \frac{w'_{\mu_i, \mathbf{c}}}{\sum_{\mathbf{c} \in \mathcal{N}(\mu_i)} w'_{\mu_i, \mathbf{c}}}, \quad (4)$$

where

$$w'_{\mu_i, \mathbf{c}} = \exp\left(-\frac{\|\mu_i - \mathbf{c}\|_2^2}{2o_c^2}\right). \quad (5)$$

Here o_c is a learnable radius. The deformed node can be computed as

$$\mathbf{c}^t = \mathbf{c} + \tilde{\mathbf{t}}_{\mathbf{c}}^t. \quad (6)$$

In cases where the 3D Gaussian exhibits anisotropy, its rotation at time t can be defined as

$$\mathbf{q}_i^t = \left(\sum_{\mathbf{c} \in \mathcal{N}(\mu_i)} w_{\mu_i, \mathbf{c}} \mathbf{r}_{\mathbf{c}}^t \right) \otimes \mathbf{q}_i, \quad (7)$$

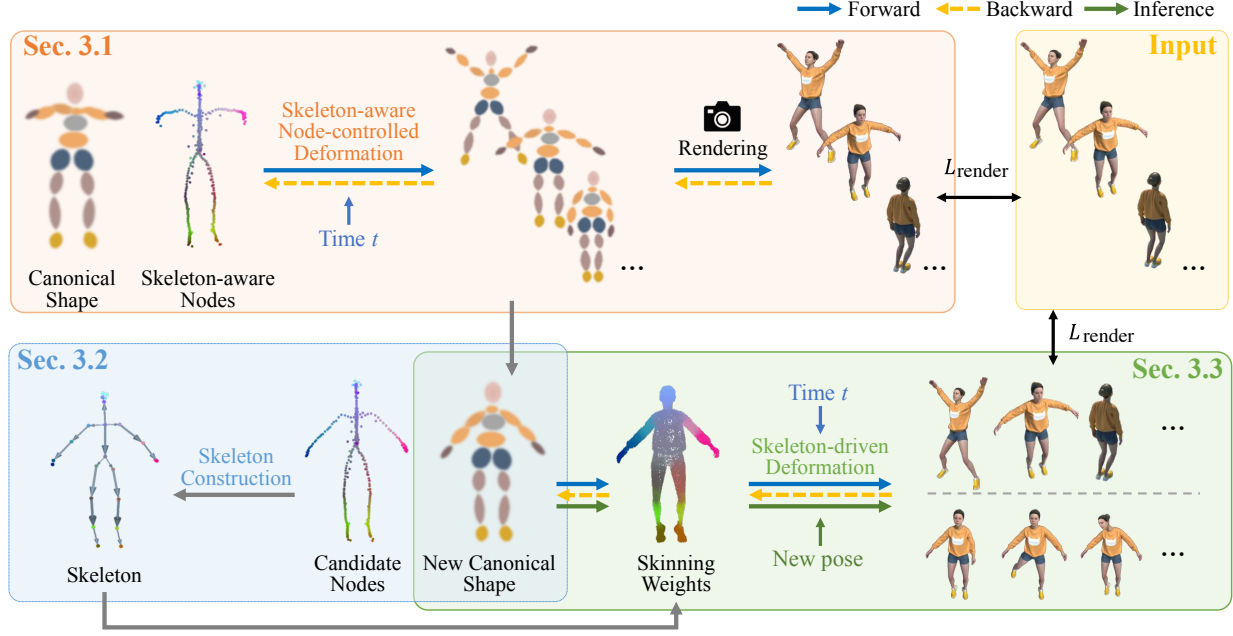


Figure 2. Overview of our RigGS. Initially, we construct a canonical 3D Gaussian, coupled with skeleton-aware node-controlled deformation, to begin the 4D reconstruction of the dynamic object. From the resulting skeleton-aware nodes, we then extract a sparse skeleton using a heuristic algorithm. Finally, leveraging the initialized deformation field and 3D Gaussians as starting values, we design learnable skinning weights and optimize a skeleton-driven deformation field. Our RigGS can be utilized for tasks such as editing, interpolation, and motion transfer, enabling real-time high-quality rendering of these new actions.

where $\mathbf{r}_c^t \in \mathbb{R}^4$ is the quaternion representation of predicted rotation; $\otimes$ is the production of quaternions; and $\hat{\mathbf{R}}_c^t, \hat{\mathbf{t}}_c^t$ and $\mathbf{r}_c^t$ will be learnable via an MLP parameterized with Θ , denoted as $F_{\Theta}(\gamma_c(\mathbf{c}), \gamma_t(t))$ with γ_c and γ_t being the positional encoding.

Loss Function for Optimizing the Initialization Stage. By capitalizing on the alignment of the skeleton with an object’s central axis, we introduce 2D skeleton projection constraints to refine the skeleton-aware nodes. Initially, we derive a set of 2D skeleton points, denoted as $\{\mathbf{p}_j^t\}$, from a reference silhouette using a skeletonize/morphological thinning algorithm [48]. For deformed nodes $\{\mathbf{c}^t\}$ at time t , we project them onto the 2D plane based on the camera viewpoint v_t and then calculate the projection loss as:

$$L_{\text{proj}}^t = \text{CD}_{\ell_1}(\text{Proj}(\mathbf{c}^t, v_t), \{\mathbf{p}_j^t\}), \quad (8)$$

where $\text{Proj}(\cdot, \cdot)$ is a projection operator, and $\text{CD}_{\ell_1}(\cdot, \cdot)$ is the ℓ_1 -norm-based Chamfer distance [5]. Notably, unlike approaches that directly extract the central axis from the reconstructed 3D model [28], our method is simpler and not reliant on the 3D reconstruction quality.

Moreover, we compute the rendering loss L_{render}^t between the rendering image $\hat{\mathbf{I}}_t$ at time t and input image $\mathbf{I}_t$ using a combination of ℓ_1 loss and D-SSIM loss [9]. Inspired by SC-GS [7], we also introduce the ARAP loss [26]

L_{arap}^t as a regularization term to maintain local rigidity during deformation.

In summary, the overall loss at time t is $L_{\text{init}}^t = L_{\text{render}}^t + w_{\text{arap}}L_{\text{arap}}^t + w_{\text{proj}}L_{\text{proj}}^t$, where w_{arap} and w_{proj} are weights to balance these terms.

3.2. Coarse-to-Fine 3D Skeleton Construction

Referring to the initial dynamic reconstruction acquired in Sec. 3.1, we begin by choosing a fresh canonical shape, followed by introducing a heuristic algorithm for deriving a sparse skeleton from the skeleton-aware nodes linked to the chosen canonical shape.

Selection of Canonical Shape. Since the canonical shape derived in Sec. 3.1 doesn’t correspond to any specific frame and hence lacks physical meaning (see Fig. 2), we replace it with a deformed shape at a chosen time representing the mean shape. Benefiting from the initialized reconstruction, we select the canonical shape based on pre-computed motion trajectories of skeleton-aware nodes. Specifically, for each node $\mathbf{c}$, we compute the mean of its trajectory, that is $\bar{\mathbf{c}} = \sum_{\mathbf{I}_t \in \mathcal{I}} \mathbf{c}^t / |\mathcal{I}|$, and select the time for the canonical shape by

$$t^* = \arg \min_t \sum_{\mathbf{c}} \|\mathbf{c}^t - \bar{\mathbf{c}}\|_2.$$

In the following, the deformed Gaussians $\{G_i^{t^*}\}$ and node $\mathbf{c}^{t^*}$ at time t^* will serve as the new canonical shape and

skeleton candidate points.

Dense Skeleton Construction. To remove redundant nodes, we initially employ farthest point sampling (FPS) on $\{\mathbf{c}^{t*}\}$ to derive a subset of nodes that are uniformly distributed $\{\mathbf{c}_s^{t*}\}_{s \in \mathcal{S}}$ with $\mathcal{S}$ being the index set. To establish the edge set to form a tree structure, we first construct an undirected fully connected graph with edge weights $\{\beta_{ij}\}_{i,j \in \mathcal{S}}$ for $\{\mathbf{c}_s^{t*}\}$. We define

$$\beta_{ij} = \sum_t d_{ij}^t / |\mathcal{I}|, \text{ where } d_{ij}^t = \|\mathbf{c}_i^t - \mathbf{c}_j^t\|,$$

considering both motion and positional information. We then use Prim’s algorithm to obtain the minimum spanning tree. This tree contains three node types: junctions (> 2 neighbors), endpoints (1 neighbor), and connection points (2 neighbors). Due to noise in $\{\mathbf{c}_s^{t*}\}$, unnecessary bifurcations may occur. Therefore, we remove redundant branches from junctions to endpoints and merge two closely located junctions. The dense skeleton tree is then constructed, denoted as $\mathcal{T}_d = \{\mathcal{J}_d, \mathcal{E}_d\}$, where $\mathcal{J}_d = \{\mathbf{J}_i\}$ denotes the node positions, and $\mathcal{E}_d$ represents the connection between the nodes.

Skeleton Simplification. For a more concise deformation representation, we devise a heuristic algorithm to obtain a sparse skeleton. We start with an empty sparse joint set, denoted as $\mathcal{J}$, and progressively incorporate probable joint nodes from $\mathcal{J}_d$. Endpoints and junctions are typically more meaningful, so we first add them into $\mathcal{J}$. We then select points with a high probability of being geometric turning points in $\mathcal{J}_d$, forming a potential joint point set $\mathcal{H}$. Using semantic labels from DINOv2 features [21], we ensure semantic symmetry by adding or removing points from $\mathcal{H}$. Finally, we incorporate $\mathcal{H}$ into $\mathcal{J}$ to form the final joint set. Using breadth-first search, we find the nearest endpoint for each junction, record the path length, and designate the junction with the longest path as the root joint $\mathbf{J}_r$. Starting from the root joint, we define the joint closer to it as the parent and the other as the child for each edge. The sparse skeleton tree is denoted as $\mathcal{T} = \{\mathcal{J}, \mathcal{A}\}$, where $\mathcal{A} = \{A_j\}_{\mathbf{J}_j \in \mathcal{J} \setminus \{\mathbf{J}_r\}}$ contains all parent indices. Fig. 3 illustrates the skeleton construction process. More details can be found in the *Supplementary Material*.

3.3. Skeleton-driven Dynamic Modeling

Built upon the initial reconstruction in Sec. 3.1, the new canonical shape and the skeleton obtained in Sec. 3.2, we establish a skeleton-driven dynamic model including an LBS-based coarse deformation with learnable skinning weights and a pose-dependent detail deformation model at this stage.

Learnable LBS-based Coarse Deformation. Denote by b_j the edge/bone between joint $\mathbf{J}_j$ and its parent $\mathbf{J}_{A_j}$. Sim-

ilar to [28, 33], we define global translation $\hat{\mathbf{t}}^t$ and rotation transformations $\{\hat{\mathbf{R}}_j^t\}_{\mathbf{J}_j \in \mathcal{J}}$ at time t , representing the global rotation for the root $\mathbf{J}_r$ and the rotation of the child joint $\mathbf{J}_j$ around the parent joint $\mathbf{J}_{A_j}$ for others. Without loss of generality, we define $\mathbf{J}_1$ as the root node. For the center of a Gaussian μ_i in the canonical shape, we can deform it using linear blend skinning (LBS) [12]:

$$\hat{\mu}_i^t = \mathbf{T}_1^t \left(\sum_{j=2}^{|\mathcal{J}|} \hat{\omega}_{i,j} \mathbf{T}_j^t \bar{\mu}_i \right), \quad (9)$$

where

$$\mathbf{T}_j^t = \mathbf{T}_{A_j}^t \hat{\mathbf{T}}_j^t \text{ and } \hat{\mathbf{T}}_j^t = \begin{bmatrix} \hat{\mathbf{R}}_j^t & \mathbf{J}_{A_j} - \hat{\mathbf{R}}_j^t \mathbf{J}_{A_j} \\ \mathbf{0} & 1 \end{bmatrix}, \quad (10)$$

and

$$\mathbf{T}_1^t = \begin{bmatrix} \hat{\mathbf{R}}_1^t & \mathbf{J}_1 - \hat{\mathbf{R}}_1^t \mathbf{J}_1 + \hat{\mathbf{t}}^t \\ \mathbf{0} & 1 \end{bmatrix}.$$

Here $\mathbf{T}_j^t$ is defined recursively by its parent $\mathbf{T}_{A_j}^t$; $\bar{\mu}_i$ is the homogeneous coordinate representation of μ_i ; $\hat{\omega}_{i,j}$ is the learnable skinning weight, which can be calculated by

$$\hat{\omega}_{i,j} = \frac{\tilde{\omega}_{i,j}}{\sum_{j=2}^{|\mathcal{J}|} \tilde{\omega}_{i,j}}, \text{ where } \tilde{\omega}_{i,j} = \eta_{i,j} \exp\left(-\frac{D^2(\mu_i, b_j)}{2\nu_j^2}\right). \quad (11)$$

Here $D(\mu_i, b_j)$ is the distance between the Gaussian center μ_i and the bone b_j ; $\{\nu_j\}_{j=2}^{|\mathcal{J}|}$ are the learnable radius parameters; $\{\eta_{i,j}\}_{j=2}^{|\mathcal{J}|}$ are the learnable scaling factors and are learned by an MLP $F_\Psi(\gamma_\eta(\mu_i))$ parameterized with Ψ ; $\{\hat{\mathbf{R}}_1^t, \dots, \hat{\mathbf{R}}_{|\mathcal{J}|}^t\}$ are represented as in quaternion form $\{\hat{\mathbf{r}}_1^t, \dots, \hat{\mathbf{r}}_{|\mathcal{J}|}^t\}$, together with translation $\hat{\mathbf{t}}^t$, learned by an MLP $F_\Phi(\gamma_t(t))$ parameterized with Φ . It can be used to correct inappropriate geometric distance-based weights by incorporating motion information. When the 3D Gaussian exhibits anisotropy, we approximate its rotation by the rotation part of $\mathbf{T}_1^t \left(\sum_{j=2}^{|\mathcal{J}|} \hat{\omega}_{i,j} \mathbf{T}_j^t \right)$.

Pose-dependent Detail Deformation. Due to the sparsity of the skeleton, the deformation field it represents exhibits local rigidity, limiting its effectiveness in areas with fine details, such as cloth wrinkles undergoing subtle deformations under physical forces. To address this, we incorporate a pose-dependent detail deformation module that learns local details. Moreover, making this module pose-related rather than time-related allows us to generate more plausible details when creating new actions. Specifically, we use an MLP parameterized with Π , denoted as $F_\Pi(\mu_i, \{\hat{\mathbf{r}}_1^t, \dots, \hat{\mathbf{r}}_{|\mathcal{J}|}^t\})$, to learn the offsets $\delta_{i,t}$ for the center position μ_i of a Gaussian. The final position of the Gaussian center is then computed as $\mu_i^t = \hat{\mu}_i^t + \delta_{i,t}$. By combining this with the rendering formula of 3D Gaussian splatting, we can obtain images from novel viewpoints and generate new motions by changing the pose.

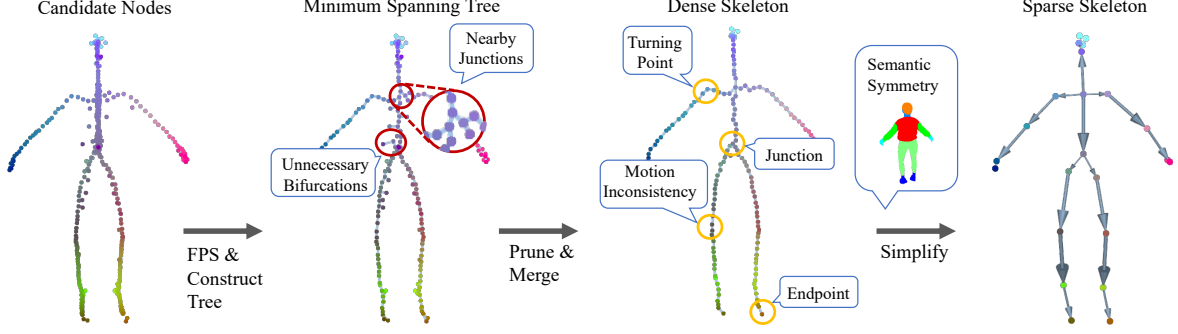


Figure 3. The process of the skeleton construction. The red circles mark the nodes that need to be removed or merged, and the yellow circles mark some locations selected as skeleton joints.

Loss Function. Apart from the rendering loss L_{render}^t detailed in Sec. 3.1, we introduce three additional loss terms, resulting in the overall loss function at time t :

$$L^t = L_{\text{render}}^t + w_{\text{proj}}^t L_{\text{proj}}^t + w_{\text{detail}}^t L_{\text{detail}}^t + w_{\text{id}} L_{\text{id}}^t.$$

To ensure the deformed skeleton remains within the deformed shape, we use a skeleton projection loss similar to Eq. (8) in Sec. 3.1. Because the skeleton joints are sparser than the skeleton-aware nodes, we sample $\mathbf{Q}_{\mathcal{T}}^t$ along the skeleton bones to compute the Chamfer distance loss:

$$L_{\text{proj}}^t = \mathcal{CD}_{\ell_1}(\text{Proj}(\mathbf{Q}_{\mathcal{T}}^t, v_t), \{\mathbf{p}_j^t\}). \quad (12)$$

Due to the imperfect accuracy of the 2D skeleton (see Fig. 5), these inaccurate 2D skeletons can lead to erroneous guidance in deformation. Therefore, we design time-specific weights: $w_{\text{proj}}^t = 10^{-3} \cdot \exp(-(L_{\text{proj}}^t)^2 / 2\xi^2)$, where ξ is half the median of $\{L_{\text{proj}}^t\}_t$. According to the 3-sigma rule of the Gaussian distribution, 2D skeletons with errors less than 1.5 times the median are considered accurate and will be subjected to stronger constraints.

To handle significant movements through LBS-based deformation and capture finer details with the other module, we introduce the regularization term

$$L_{\text{detail}}^t = \sum_i \|\delta_{i,t}\|_2^2 / |\mathcal{G}|, \quad (13)$$

aimed at minimizing detailed deformations, where $|\mathcal{G}|$ denotes the number of Gaussians. The weight w_{detail}^t is set to 10^3 when $t = t^*$ and 1 otherwise, ensuring consistency between shapes in the canonical shape and at the selected moment t^* . Additionally, we enforce a constraint to align the skeleton pose at t^* towards an identity transformation. So we define: $L_{\text{id}}^t = \sum_{j=1}^{|\mathcal{J}|} \|\hat{\mathbf{r}}_j - \mathbf{I}_q\|_2^2 / |\mathcal{J}|$ for $t = t^*$ and $L_{\text{id}}^t = 0$ for others, where $\mathbf{I}_q$ is the identity quaternion.

4. Results

4.1. Experimental Settings

Implementation Details. In our RigGS, we utilized MLPs with 8 linear layers with a feature dimension of 256 to implement $F_{\Phi}(\cdot)$, $F_{\Psi}(\cdot)$, $F_{\Theta}(\cdot)$, and $F_{\Pi}(\cdot)$. The ADAM optimizer [3] was adopted during training. More details are available in the *Supplementary Material*. We conducted all experiments on a single NVIDIA RTX A6000 GPU.

Datasets. We used two synthetic datasets: the D-NeRF dataset [23], which includes 8 sequences, and the DG-Mesh dataset [14], which includes 6 sequences. Each dataset contains a series of continuous actions. However, the ‘‘Bouncing balls’’ in the D-NeRF dataset and ‘‘Torus2sphere’’ in the DG-Mesh dataset do not match our task setting, and the test view frames in ‘‘Lego’’ in the D-NeRF dataset do not align with the training actions, as noted in [7]. Therefore, we tested only the remaining 6 sequences from the D-NeRF dataset and 5 sequences from the DG-Mesh dataset. Additionally, we considered real-captured datasets: 6 subjects (377, 386, 387, 392, 393, 394) on the ZJU-MoCap dataset [22].

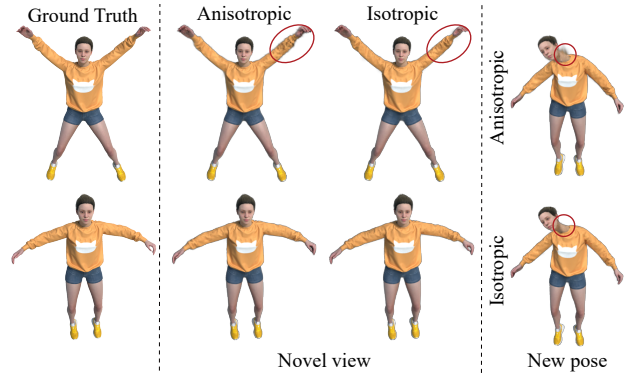


Figure 4. Novel view rendering and repose results via anisotropic or isotropic 3D Gaussians.

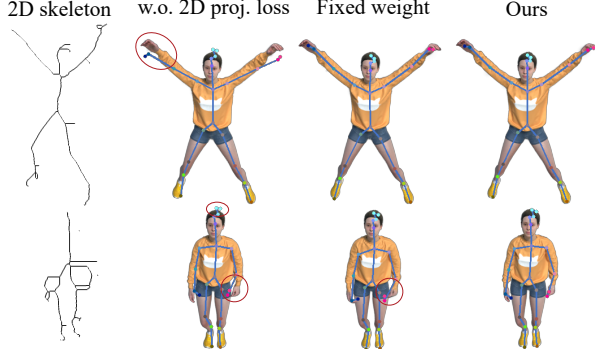


Figure 5. Comparison of visual results for different variants on 2D projection loss term.

4.2. Ablation Study

On the D-NeRF dataset, we evaluated the effectiveness of each component of our method using the following variations:

- *Anisotropic or isotropic 3D Gaussians.* Although anisotropic 3D Gaussians yield images closer to the ground truth, they suffer significant quality degradation when the new poses differ greatly from the training pose, as shown in Fig. 4. Therefore, we use the isotropic 3D Gaussians for all experiments.
- *Without 2D projection loss L_{proj}^t or fixed weight w_{proj}^t during training the skeleton-driven dynamic model.* The 2D projection loss L_{proj}^t assesses how well the skeleton aligns with the 3D Gaussians. We tested performances without this loss and with fixed weights, setting $w_{proj}^t = 10^{-3}$. Fig. 5 shows that without the projection loss, the skeleton cannot be embedded into the 3D Gaussians. Using fixed w_{proj}^t instead of adaptive weights can cause skeletons to protrude beyond the shape in some frames due to inaccuracies in the extracted 2D skeletons.
- *Skeleton construction and skeleton-driven deformation field.* More discussions, numerical results, and visual results are available in the *Supplementary Material*.

4.3. Comparisons

Novel View Synthesis. To demonstrate the advantages of our RigGS, we compared it with state-of-the-art approaches: D-NeRF [23], TiNeuVox [6], 4D-GS [32], SC-GS [7], and AP-NeRF [28] in novel viewpoint synthesis¹. Except for AP-NeRF, other methods do not bind the skeleton, and *do not* support editing the object through repose. As shown in Table 1, our rendering accuracy is slightly lower than SC-GS but significantly higher than the other methods. Besides the use of anisotropic 3D Gaussians, SC-GS achieves better detail fitting due to its more freedom of deformation field (512 control points) and tempo-

¹Since D-NeRF failed to run on the "beagle" sequence on DG-Mesh dataset, the average results of the other four datasets are presented.

Table 1. Comparisons of the average precision (PSNR $\uparrow$ / SSIM $\uparrow$ / LPIPS $\downarrow$) on the D-NeRF dataset and DG-Mesh dataset.

Method	D-NeRF [23]	DG-Mesh [14]
	PSNR / SSIM / LPIPS	PSNR / SSIM / LPIPS
D-NeRF [23]	30.48 / 0.973 / 0.0492	28.17 / 0.957 / 0.0778
TiNeuVox [6]	32.60 / 0.983 / 0.0436	31.95 / 0.967 / 0.0477
4D-GS [32]	33.25 / 0.989 / 0.0233	33.96 / 0.979 / 0.0272
SC-GS [7]	43.04 / 0.998 / 0.0066	38.96 / 0.993 / 0.0136
AP-NeRF [28]	30.94 / 0.970 / 0.0350	31.83 / 0.967 / 0.0460
Ours	<u>40.82 / 0.996 / 0.0112</u>	<u>37.65 / 0.991 / 0.0169</u>

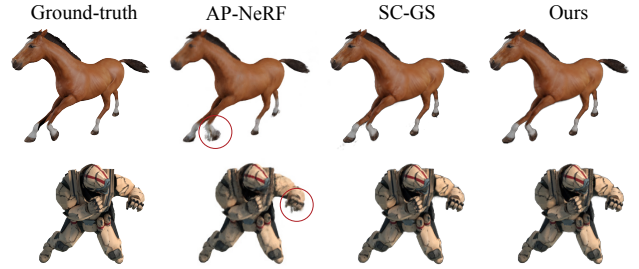


Figure 6. Comparison with state-of-the-art methods for novel view synthesis.

Table 2. Comparisons of the average precision on ZJU-MoCap.

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
AP-NeRF [28]	25.62	0.919	0.0934
Ours	33.54	0.975	0.0327

rally adjustments in scales and rotations of 3D Gaussians. However, these configurations do not apply to our task because we require sparse skeletons and require all deformation variables to be pose-related for generating plausible movements. Fig. 6 shows that visually, our method is comparable to SC-GS without significant disadvantages. Additionally, we also compared our method with AP-NeRF on the real-captured ZJU-MoCap dataset, and show the numerical results in Table 2. More visual results are available in the *Supplementary Material*.

Skeleton and Skinning Weights. Closely related to our research, AP-NeRF [28] stands as an automated framework designed for the construction of skeletons and the modeling

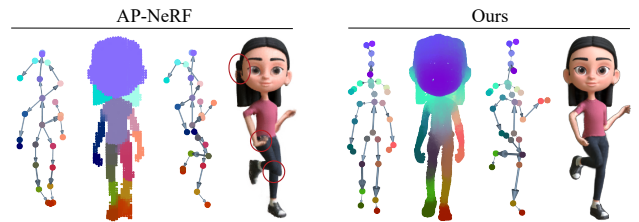


Figure 7. Comparisons of the skeleton, the skinning weights, edited skeleton pose, and the corresponding reposed objects.

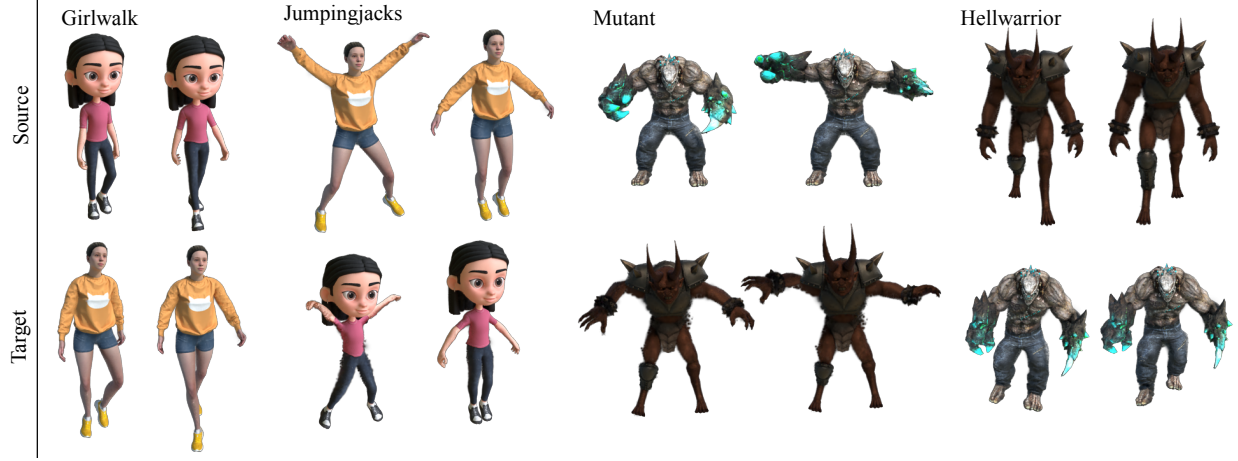


Figure 8. Visualization of motion transfer from the source object to the target object.

of articulated objects. Initialized with TiNeuVox [6], it utilizes reconstructed surface points to extract a 3D skeleton employing the Medial Axis Transform (MAT), refining the skeleton progressively through optimization steps. In contrast, our approach diminishes the dependency on the quality of 3D reconstruction. As shown in Fig. 7, we compared AP-NeRF with our method on the constructed skeleton and skinning weights. Furthermore, we manipulated the poses of the skeletons to generate the same new actions for the object. We can observe that our skeleton is more reasonable, and the skinning weights are smoother, avoiding discontinuities at the joints. In addition, we have higher rendering quality.

Editing and Interpolation. Thanks to our skeleton-driven deformation field, we can create new motions for the object by editing the rotations of the skeleton bones. To facilitate this process, we have developed a GUI that allows for interactive editing and real-time rendering of the results. Figs. 1, 7 and 9 display several editing performances. Additionally, as shown in Fig. 9, we can easily interpolate between two poses of an object in a plausible manner.

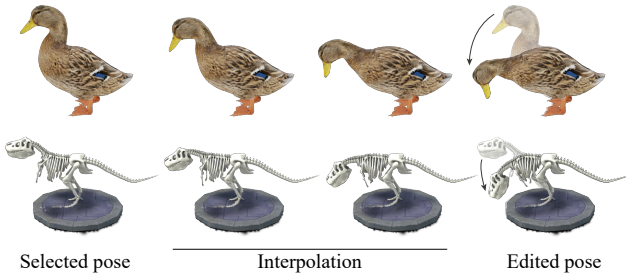


Figure 9. Visualizations of the selected pose, the edited pose, and the interpolated results between them.

Motion Transfer. Our method can also be utilized for motion transfer tasks. As illustrated in Fig. 8, for two sequences with similar structures, we first manually annotate the correspondences between their skeletons, repose the target object to match the source object’s pose, and then transfer the source object’s pose sequences to drive the target object, generating new motion sequences.

5. Conclusion and Discussion

We have presented RigGS, a skeleton-driven modeling approach that reconstructs articulated objects from 2D videos without relying on any template priors. First, we initialized the reconstruction of dynamic objects from input videos using a skeleton-aware node-controlled deformation field combined with a canonical 3D Gaussian representation, which also yields candidate points for the skeleton. Second, we introduced an automated skeleton extraction algorithm to obtain a sparse skeleton from these candidate points. Finally, we established a skeleton-driven dynamic modeling approach that binds the canonical 3D Gaussians with the skeleton, enabling tasks such as editing, interpolation, and motion transfer, while rendering high-quality images from novel viewpoints. Experimental results demonstrated that our method achieves rendering quality close to state-of-the-art novel view synthesis methods while easily generating new poses for the objects.

While our RigGS has shown promising results in many cases, its effectiveness may be limited when dealing with sparse viewpoints, inaccurate estimation of camera poses, or excessive motion. The pose-related appearance is also not modeled. More useful techniques for 3D reconstruction and editing will be helpful in solving these challenging problems. Additionally, exploring more complex input signals, such as text and images, for semantic skeleton pose editing could be an interesting future direction.

References

- [1] Jinseok Bae, Hojun Jang, Cheol-Hui Min, Hyungun Choi, and Young Min Kim. Neural marionette: Unsupervised learning of motion skeleton and latent dynamics from volumetric video. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 86–94, 2022. 2
- [2] Aljaž Božič, Pablo Palafox, Michael Zollhöfer, Justus Thies, Angela Dai, and Matthias Nießner. Neural deformation graphs for globally-consistent non-rigid reconstruction. In *IEEE Conf. Comput. Vis. Pattern Recog.*, 2021. 2
- [3] P Kingma Diederik. Adam: A method for stochastic optimization. (*No Title*), 2014. 6
- [4] Zhiyang Dou, Cheng Lin, Rui Xu, Lei Yang, Shiqing Xin, Taku Komura, and Wenping Wang. Coverage axis: Inner point selection for 3d shape skeletonization. In *Comput. Graph. Forum*, pages 419–432. Wiley Online Library, 2022. 2
- [5] Haoqiang Fan, Hao Su, and Leonidas J Guibas. A point set generation network for 3d object reconstruction from a single image. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 605–613, 2017. 4
- [6] Jiemin Fang, Taoran Yi, Xinggang Wang, Lingxi Xie, Xiaopeng Zhang, Wenyu Liu, Matthias Nießner, and Qi Tian. Fast dynamic radiance fields with time-aware neural voxels. In *SIGGRAPH Asia 2022 Conference Papers*, 2022. 7, 8
- [7] Yi-Hua Huang, Yang-Tian Sun, Ziyi Yang, Xiaoyang Lyu, Yan-Pei Cao, and Xiaojuan Qi. Sc-gs: Sparse-controlled gaussian splatting for editable dynamic scenes. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 4220–4230, 2024. 2, 4, 6, 7
- [8] Boyi Jiang, Yang Hong, Hujun Bao, and Juyong Zhang. Selfrecon: Self reconstruction your digital avatar from monocular video. In *IEEE Conf. Comput. Vis. Pattern Recog.*, 2022. 2
- [9] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023. 4
- [10] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023. 2, 3
- [11] Tianshu Kuai, Akash Karthikeyan, Yash Kant, Ashkan Mirzaei, and Igor Gilitschenski. Camm: Building category-agnostic and animatable 3d models from monocular videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 6586–6596, 2023. 2, 3
- [12] J. P. Lewis, Matt Corder, and Nickson Fong. Pose space deformation: a unified approach to shape interpolation and skeleton-driven deformation. In *Proceedings of the 27th Annual Conference on Computer Graphics and Interactive Techniques*, page 165–172, USA, 2000. ACM Press/Addison-Wesley Publishing Co. 5
- [13] Cheng Lin, Changjian Li, Yuan Liu, Nenglu Chen, Yi-King Choi, and Wenping Wang. Point2skeleton: Learning skeletal representations from point clouds. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 4277–4286, 2021. 2
- [14] Isabella Liu, Hao Su, and Xiaolong Wang. Dynamic gaussians mesh: Consistent mesh reconstruction from monocular videos. In *Int. Conf. Learn. Represent.*, 2025. 6, 7
- [15] Qingming Liu, Yuan Liu, Jiepeng Wang, Xianqiang Lyv, Peng Wang, Wenping Wang, and Junhui Hou. Modgs: Dynamic gaussian splatting from casually-captured monocular videos. In *Int. Conf. Learn. Represent.*, 2025. 2
- [16] Shaowei Liu, Saurabh Gupta, and Shenlong Wang. Building rearticulable models for arbitrary 3d objects from 4d point clouds. In *IEEE Conf. Comput. Vis. Pattern Recog.*, 2023. 3
- [17] Matthew Loper, Naureen Mahmood, Javier Romero, Gerard Pons-Moll, and Michael J. Black. Smpl: a skinned multi-person linear model. *ACM Trans. Graph.*, 34(6), 2015. 1, 2
- [18] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *Eur. Conf. Comput. Vis.*, 2020. 2
- [19] GART: Gaussian Articulated Template Models. Jiahui lei and yufu wang and georgios pavlakos and lingjie liu and kostas daniilidis. In *IEEE Conf. Comput. Vis. Pattern Recog.*, 2024. 2
- [20] Gyeongsik Moon, Takaaki Shiratori, and Shunsuke Saito. Expressive whole-body 3D gaussian avatar. In *Eur. Conf. Comput. Vis.*, 2024. 2
- [21] Maxime Oquab, Timothée Darcet, Theo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Russell Howes, Po-Yao Huang, Hu Xu, Vasu Sharma, Shangwen Li, Wojciech Galuba, Mike Rabbat, Mido Assran, Nicolas Ballas, Gabriel Synnaeve, Ishan Misra, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision, 2023. 5
- [22] Sida Peng, Yuanqing Zhang, Yinghao Xu, Qianqian Wang, Qing Shuai, Hujun Bao, and Xiaowei Zhou. Neural body: Implicit neural representations with structured latent codes for novel view synthesis of dynamic humans. In *IEEE Conf. Comput. Vis. Pattern Recog.*, 2021. 6
- [23] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. D-nerf: Neural radiance fields for dynamic scenes. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 10318–10327, 2021. 6, 7
- [24] Javier Romero, Dimitrios Tzionas, and Michael J. Black. Embodied hands: Modeling and capturing hands and bodies together. *ACM Trans. Graph.*, 36(6), 2017. 1, 2
- [25] Chaoyue Song, Jiacheng Wei, Chuan Sheng Foo, Guosheng Lin, and Fayao Liu. Reacto: Reconstructing articulated objects from a single video. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 5384–5395, 2024. 3
- [26] Olga Sorkine and Marc Alexa. As-rigid-as-possible surface modeling. In *Symposium on Geometry processing*, pages 109–116. Citeseer, 2007. 4
- [27] Tao Tu, Ming-Feng Li, Chieh Hubert Lin, Yen-Chi Cheng, Min Sun, and Ming-Hsuan Yang. Dreamo: Articulated 3d

- reconstruction from a single casual video. *arXiv preprint arXiv:2312.02617*, 2023. 3
- [28] Lukas Uzolas, Elmar Eisemann, and Petr Kellnhofer. Template-free articulated neural point clouds for reposable view synthesis. *Adv. Neural Inform. Process. Syst.*, 36, 2024. 2, 3, 4, 5, 7
- [29] Diwen Wan, Yuxiang Wang, Ruijie Lu, and Gang Zeng. Template-free articulated gaussian splatting for real-time reposable dynamic view synthesis. In *Adv. Neural Inform. Process. Syst.*, 2024. 3
- [30] Peng Wang, Lingjie Liu, Yuan Liu, Christian Theobalt, Taku Komura, and Wenping Wang. Neus: Learning neural implicit surfaces by volume rendering for multi-view reconstruction. *Adv. Neural Inform. Process. Syst.*, 2021. 2
- [31] Zimeng Wang, Zhiyang Dou, Rui Xu, Cheng Lin, Yuan Liu, Xiaoxiao Long, Shiqing Xin, Taku Komura, Xiaoming Yuan, and Wenping Wang. Coverage axis++: Efficient inner point selection for 3d shape skeletonization. In *Comput. Graph. Forum*, page e15143. Wiley Online Library, 2024. 2
- [32] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4d gaussian splatting for real-time dynamic scene rendering. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 20310–20320, 2024. 2, 7
- [33] Shangzhe Wu, Ruining Li, Tomas Jakab, Christian Rupprecht, and Andrea Vedaldi. Magicpony: Learning articulated 3d animals in the wild. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 8792–8802, 2023. 2, 5
- [34] Yuefan Wu*, Zeyuan Chen*, Shaowei Liu, Zhongzheng Ren, and Shenlong Wang. CASA: Category-agnostic skeletal animal reconstruction. In *Adv. Neural Inform. Process. Syst.*, 2022. 2
- [35] Zhan Xu, Yang Zhou, Evangelos Kalogerakis, and Karan Singh. Predicting animation skeletons for 3d articulated models via volumetric nets. In *2019 international conference on 3D vision (3DV)*, pages 298–307. IEEE, 2019. 2, 3
- [36] Zhan Xu, Yang Zhou, Evangelos Kalogerakis, Chris Landreth, and Karan Singh. Rignet: Neural rigging for articulated characters. *ACM Trans. Graph.*, 39(4), 2020. 2, 3
- [37] Zhan Xu, Yang Zhou, Li Yi, and Evangelos Kalogerakis. Morig: Motion-aware rigging of character meshes from point clouds. In *SIGGRAPH Asia 2022 conference papers*, pages 1–9, 2022. 2
- [38] Gengshan Yang, Minh Vo, Natalia Neverova, Deva Ramanan, Andrea Vedaldi, and Hanbyul Joo. Banmo: Building animatable 3d neural models from many casual videos. In *IEEE Conf. Comput. Vis. Pattern Recog.*, 2022. 2
- [39] Gengshan Yang, Chaoyang Wang, N. Dinesh Reddy, and Deva Ramanan. Reconstructing animatable categories from videos. In *IEEE Conf. Comput. Vis. Pattern Recog.*, 2023. 2
- [40] Gengshan Yang, Chaoyang Wang, N Dinesh Reddy, and Deva Ramanan. Reconstructing animatable categories from videos. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 16995–17005, 2023. 2
- [41] Ziyi Yang, Xinyu Gao, Wen Zhou, Shaohui Jiao, Yuqing Zhang, and Xiaogang Jin. Deformable 3d gaussians for high-fidelity monocular dynamic scene reconstruction. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 20331–20341, 2024. 2
- [42] Zhangsihao Yang, Mingyuan Zhou, Mengyi Shan, Bingbing Wen, Ziwei Xuan, Mitch Hill, Junjie Bai, Guo-Jun Qi, and Yalin Wang. Omnimotiongpt: Animal motion generation with limited data, 2024. 2
- [43] Chun-Han Yao, Wei-Chih Hung, Yuanzhen Li, Michael Rubinstein, Ming-Hsuan Yang, and Varun Jampani. LASSIE: learning articulated shapes from sparse image ensemble via 3d part discovery. In *Adv. Neural Inform. Process. Syst.*, 2022. 2
- [44] Yuxin Yao, Siyu Ren, Junhui Hou, Zhi Deng, Juyong Zhang, and Wenping Wang. Dynosurf: Neural deformation-based temporally consistent dynamic surface reconstruction. In *Eur. Conf. Comput. Vis.*, 2024. 2
- [45] Meng You and Junhui Hou. Decoupling dynamic monocular videos for dynamic view synthesis. *IEEE Trans. Vis. Comput. Graph.*, 2024. 2
- [46] Meng You, Zhiyu Zhu, Hui Liu, and Junhui Hou. Nvs-solver: Video diffusion model as zero-shot novel view synthesizer. In *Int. Conf. Learn. Represent.*, 2025. 2
- [47] Tingyang Zhang, Qingzhe Gao, Weiyu Li, Libin Liu, and Baoquan Chen. Bags: Building animatable gaussian splatting from a monocular video with diffusion priors, 2024. 2
- [48] T. Y. Zhang and Ching Y. Suen. A fast parallel algorithm for thinning digital patterns. *Commun. ACM*, 27(3):236–239, 1984. 4
- [49] Silvia Zuffi, Angjoo Kanazawa, David Jacobs, and Michael J. Black. 3D menagerie: Modeling the 3D shape and pose of animals. In *IEEE Conf. Comput. Vis. Pattern Recog.*, 2017. 1, 2